

Case Study Artefact Walkthrough

Simon Knight

March 2026 • Version 1.0

Last updated: March 17, 2026

Abstract. This document provides a detailed walkthrough of the four case study artefacts from Knight's PhD thesis (Chapter 8), ported to the `ank_netcfg` Rust pipeline. For each case study, we show the input topology, the complete blueprint, the intermediate representations produced at each pipeline stage, and the rendered device configurations. Large artefacts are truncated with ellipsis markers. This document serves as a companion to NETCFG-TR-001 Appendix A.

Contents

1	Introduction	2
1.1	Pipeline Architecture	2
2	CS1: Simplified Small Internet	2
2.1	Topology Diagram.	2
2.2	Topology Input	2
2.3	Blueprint	4
2.4	Assertions	5
2.5	Layer Progression	6
2.6	Intermediate Representation: DeviceContext	6
2.7	Stanza Output	7
2.8	Rendered Configuration	8
2.9	Cross-Reference: AutoNetkit Thesis Output	9
3	CS2: Complete Small Internet	10
3.1	Topology Differences from CS1	10
3.2	Policy Layer.	11
3.3	Policy Stanza Output.	12
3.4	Rendered Policy Configuration.	13
3.5	Documented Gaps.	14
4	CS3: VLAN Campus Network	14
4.1	Campus Topology Diagram	14
4.2	Topology Input	14
4.3	Blueprint: VLAN Tagging via tag_nodes	15
4.4	Mixed Device Output	16
5	CS4: Large-Scale NREN	16
5.1	GraphML Conversion	16
5.2	Hub-and-Spoke Mesh for Route Reflectors	17
5.3	Placeholder Topology (14-node).	18
5.4	Route-Reflector Stanza Output.	18
6	The NTE-QL Query Language	19
6.1	Selector Syntax	19
6.2	Context Variables	19
6.3	Group Expansion.	20
6.4	Advanced Features	20
7	Running the Pipeline	21
8	Diff Engine and Mutation Plans	23
8.1	Change Types	23
8.2	Diff Algorithm.	23
9	Primitive Algorithms	24
9.1	Mesh Nodes: Full Mesh vs Hub-and-Spoke	24
9.2	IPAM: The Five-Pass Allocation Algorithm	25
9.3	Protocol Layer: Clone-and-Merge	26
10	Typed Intermediate Representation	26
10.1	StandardStanza Variants	26
10.2	Protocol Overlay Schemas	27
10.3	Assertion Check Types	28
11	Security and QoS Intermediate Representations	28
11.1	Security Model	28

11.2	QoS Model	29
12	Layer Execution Model	30
13	DeviceContext: The Per-Node State Store	30
13.1	Field Map	30
13.2	Data Flow	31
14	Topology Import Pipeline	32
14.1	Topogen Import Algorithm	32
15	Vendor Template Comparison	33
15.1	Template Coverage Matrix	34
16	Blast Radius and Hardware Constraints	34
16.1	Blast Radius Guards	34
16.2	Hardware Library	35
17	Pipeline Constraints Discovered	36
18	Compilation Metrics	36
18.1	Per-Layer Statistics	37
19	Lessons Learned	37
19.1	What Transferred Cleanly	37
19.2	What Required Rethinking	37
19.3	Recommendations for Future Primitives	38
19.4	Validation Strategy	38
20	Summary	38

Contents

1 Introduction

The `ank_netcfg` pipeline compiles a *design plan* (blueprint) against an imported *topology* through a sequence of *layers*, each applying one or more *primitives* that mutate the graph. The pipeline stages are:

1. **Import** — parse a topogen-format YAML file into an NTE archive (graph + Parquet DataFrames).
2. **Layer evaluation** — traverse layers in dependency order, applying primitives (mesh_nodes, provision_ips, build_protocol_layer, etc.).
3. **Stanza generation** — each primitive emits untyped Stanza records (kind, key, fields) attached to device contexts.
4. **Target transform** — untyped stanzas are converted to typed StandardStanza variants (e.g. BgpNeighbor, OspfNeighbor, Interface).
5. **Template rendering** — MiniJinja vendor templates (`cisco.j2`, `eos.j2`, etc.) consume the typed stanzas and emit per-device configuration files.

This document walks through each case study showing concrete artefacts at every stage. The four case studies are:

CS1 Simplified Small Internet — 14 routers, 7 ASes, OSPF + iBGP + eBGP

CS2 Complete Small Internet — adds BGP policy (route-maps, prefix-lists, safety filters)

CS3 VLAN Campus Network — 9 routers + 5 switches, exposes L2 gap

CS4 Large-Scale NREN — scale benchmark with route-reflector hierarchy

1.1 Pipeline Architecture

Figure 1 shows the five-stage compilation pipeline from YAML input to rendered device configuration.

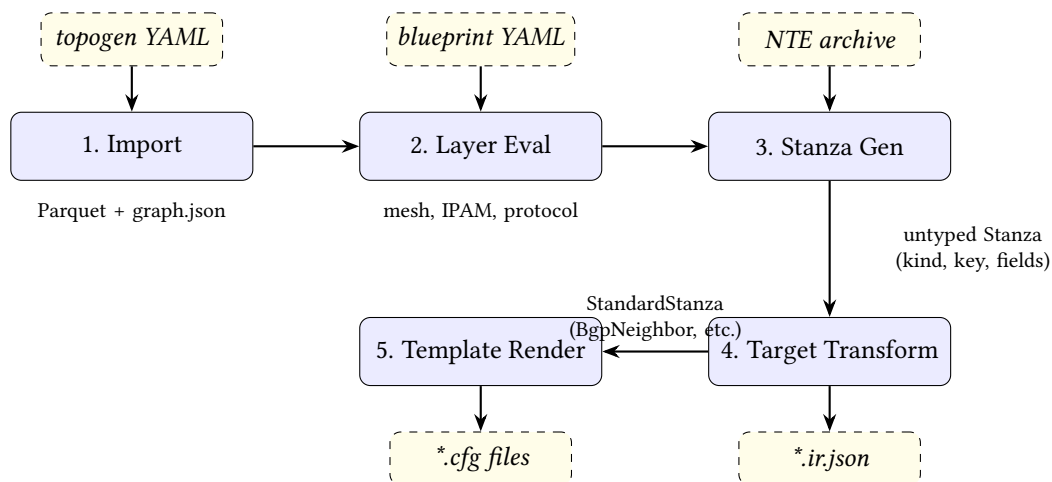


Figure 1. Five-stage compilation pipeline. Boxes are stages; dashed boxes are artefact files.

2 CS1: Simplified Small Internet

2.1 Topology Diagram

Figure 2 shows the 14-router, 7-AS topology. Solid lines are intra-AS edges (full-mesh within multi-router ASes); dashed lines are inter-AS peering links.

2.2 Topology Input

The input topology is a topogen-format YAML file defining 14 routers across 7 autonomous systems with 17 edges (9 intra-AS + 8 inter-AS).

```
version: "1.0"
name: "cs1-simplified-small-internet"
```

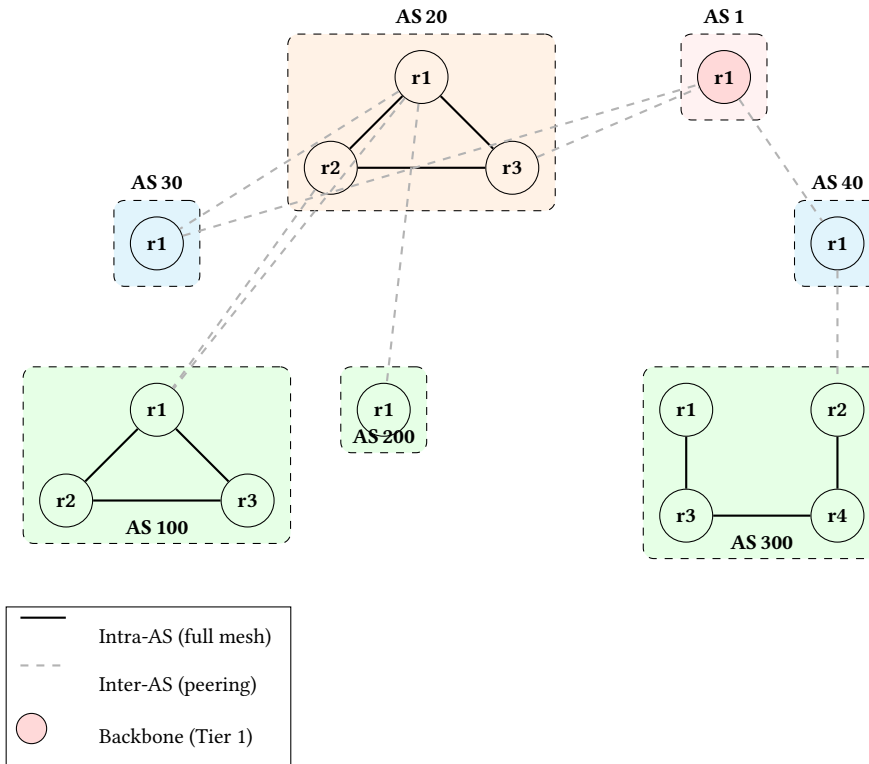


Figure 2. CS1 topology: 14 routers across 7 autonomous systems. Multi-router ASes (20, 100, 300) have intra-AS full-mesh edges. Eight inter-AS peering links connect the ASes.

nodes:

```
# AS1 - Backbone / Tier 1
- id: "as1r1"
  node_type: "Router"
  properties:
    asn: 1
    role: "backbone"
    as_group: "as1"
    device_os: "cisco"      # required for template rendering

# AS20 - Provider / Tier 2
- id: "as20r1"
  node_type: "Router"
  properties:
    asn: 20
    role: "border"
    as_group: "as20"
    device_os: "cisco"
- id: "as20r2"
  node_type: "Router"
  properties: { asn: 20, role: "border", as_group: "as20", device_os: "cisco" }
- id: "as20r3"
  node_type: "Router"
  properties: { asn: 20, role: "border", as_group: "as20", device_os: "cisco" }

# ... AS30, AS40 (1 router each), AS100 (3 routers),
#       AS200 (1 router), AS300 (4 routers) ...
```

Edge definitions include both intra-AS and inter-AS links:

edges:

```
# Intra-AS20 (full mesh: r1-r2, r2-r3, r1-r3)
- source_id: "as20r1"
```

```

    target_id: "as20r2"
    properties: { link_type: "intra-as" }
- source_id: "as20r2"
  target_id: "as20r3"
  properties: { link_type: "intra-as" }
- source_id: "as20r1"
  target_id: "as20r3"
  properties: { link_type: "intra-as" }

# Inter-AS peering
- source_id: "as20r2"
  target_id: "as100r1"
  properties: { link_type: "inter-as" }
- source_id: "as200r1"
  target_id: "as20r1"
  properties: { link_type: "inter-as" }
# ... 6 more inter-AS edges ...

```

After import, nodes are stored in the NTE archive as a Parquet DataFrame (`Router.parquet`) with columns: `id`, `asn`, `role`, `as_group`. Each node is assigned to the physical layer. Edge properties (`link_type`) are *not* preserved in the current import path — this is a known limitation.

2.3 Blueprint

The blueprint defines six layers with explicit dependency ordering:

```

version: 1
layers:
- name: pools
  requires: []
  primitives:
    - type: global_pool
      name: infra-v4
      pool: "192.168.0.0/16"
      vrf: null
    - type: global_pool
      name: loopback-v4
      pool: "10.0.0.0/16"
      vrf: null

- name: physical
  requires: [pools]
  primitives:
    - type: mesh_nodes
      selector: $as20_routers    # expands to: nodes[as_group == 'as20']
      mesh_type: full
    - type: mesh_nodes
      selector: $as100_routers
      mesh_type: full
    - type: mesh_nodes
      selector: $as300_routers
      mesh_type: full

- name: addressing
  requires: [physical]
  primitives:
    - type: provision_ips
      selector: edges[true]
      pool: infra-v4
      subnet_size: 30

```

```
strategy: dense
```

```
- name: ospf_overlay
  requires: [addressing]
  primitives:
    - type: build_protocol_layer
      selector: $as20_routers
      layer: ospf
      config: { area: "0.0.0.0", protocol_type: ospf }
      clone_underlying: true
    - type: build_protocol_layer
      selector: $as100_routers
      layer: ospf
      config: { area: "0.0.0.0", protocol_type: ospf }
      clone_underlying: true
    - type: build_protocol_layer
      selector: $as300_routers
      layer: ospf
      config: { area: "0.0.0.0", protocol_type: ospf }
      clone_underlying: true

- name: ibgp_overlay
  requires: [ospf_overlay]
  primitives:
    - type: build_protocol_layer
      selector: $as20_routers
      layer: bgp
      config: { asn_base: "20", protocol_type: bgp }
      clone_underlying: true
      # ... AS100 (asn_base: "100"), AS300 (asn_base: "300") ...

- name: ebgp_overlay
  requires: [ibgp_overlay]
  primitives:
    - type: build_protocol_layer
      selector: $border_routers
      layer: bgp
      config: { protocol_type: bgp }
      clone_underlying: true
```

Group definitions. Selectors reference named groups, expanded before evaluation:

```
groups:
  all_routers: nodes[true]
  border_routers: nodes[role == 'border']
  as20_routers: nodes[as_group == 'as20']
  as100_routers: nodes[as_group == 'as100']
  as300_routers: nodes[as_group == 'as300']
  # ... as1, as30, as40, as200 ...
```

2.4 Assertions

Assertions are evaluated after all layers complete. CS1 defines six:

```
assertions:
- name: ospf-as20-connected
  severity: error
  select: "nodes[as_group == 'as20']"
```

```

check: { type: is_connected }
help: "AS20 OSPF subgraph is disconnected."

- name: all-routers-present
  severity: error
  select: "nodes[true]"
  check: { type: min_count, count: 14 }
  help: "Expected 14 routers in topology."

- name: edges-have-src-ip
  severity: error
  select: "edges[true]"
  check: { type: field_exists, field: src_ip }
  help: "An edge is missing its IPv4 source address."

- name: as300-reaches-as100
  severity: warning # warning: fails in validate mode (read-only)
  select: "nodes[as_group == 'as300']"
  check:
    type: reachability
    target_selector: "nodes[as_group == 'as100']"

```

Pipeline finding. The assertion select field does *not* expand \$group references. All assertions must use inline selectors (e.g. nodes[as_group == 'as20']) rather than \$as20_routers.

2.5 Layer Progression

Table 1 summarises what each layer adds to the topology graph.

Table 1. CS1 layer progression: graph mutations per layer.

Layer	Primitives	Effect
pools	global_pool ×2	Registers infra-v4 (192.168.0.0/16) and loopback-v4 (10.0.0.0/16) address pools.
physical	mesh_nodes ×3	Creates full-mesh edges within AS20 (3 nodes), AS100 (3 nodes), and AS300 (4 nodes). Single-router ASes (1, 30, 40, 200) need no mesh. Nodes must be in the physical layer to be found by selectors.
addressing	provision_ips	Allocates /30 subnets from infra-v4 to all edges. Each edge endpoint receives src_ip and dst_ip. Populates Interface stanzas with addresses.
ospf_overlay	build_protocol_layer ×3	Creates OSPF protocol layer for AS20, AS100, AS300. Clones underlying edges. Emits OspfNeighbor stanzas per interface.
ibgp_overlay	build_protocol_layer ×3	Creates iBGP sessions within each multi-router AS. Uses full-mesh edges from physical. Emits BgpNeighbor stanzas.
ebgp_overlay	build_protocol_layer	Creates eBGP sessions on inter-AS links between border routers. Emits BgpNeighbor stanzas with differing local/remote ASNs.

2.6 Intermediate Representation: DeviceContext

After IPAM and protocol layer evaluation, each node carries a DeviceContext with allocated addresses, peer mappings, and protocol state. The following shows the *expected* IR for as20r1 (a border router in AS20 with 5 peers) when the single-layer pipeline is used (see Section 7 for actual multi-layer output):


```
{
  "hostname": "as20r1",
  "device_os": "cisco",
  "role": "border",
  "as_group": "as20",
  "asn": 20,
  "mesh_type": "full",
  "peer_count": 5,
  "peer_interfaces": {
    "as20r2": "Ethernet0",
    "as20r3": "Ethernet1",
    "as100r1": "Ethernet2",
    "as200r1": "Ethernet3",
    "as30r1": "Ethernet4"
  },
  "src_ip": "192.168.0.1",
  "dst_ip": "192.168.0.2",
  "subnet": "192.168.0.0/30",
  "vrf": "default"
}
```

2.7 Stanza Output

The pipeline emits typed StandardStanza records per device. For as20r1, the stanzas include:

Interface stanzas (one per peer link):

```
{
  "kind": "interface",
  "key": "Ethernet0",
  "fields": {
    "name": "Ethernet0",
    "address": "192.168.0.1/30",
    "description": "to as20r2",
    "enabled": true,
    "mtu": 1500,
    "vrf": "default"
  }
}
```

OSPF neighbour stanzas (one per intra-AS interface):

```
{
  "kind": "ospf_neighbor",
  "fields": {
    "interface": "Ethernet0",
    "area": "0.0.0.0"
  }
}
```

BGP neighbour stanzas (iBGP peers within AS20 + eBGP to other ASes):

```
{
  "kind": "bgp_neighbor",
  "fields": {
    "peer_ip": "192.168.0.2",

```

```

    "remote_as": 20,
    "local_as": 20,
    "description": "iBGP to as20r2"
  }
}

```

```

{
  "kind": "bgp_neighbor",
  "fields": {
    "peer_ip": "192.168.0.9",
    "remote_as": 100,
    "local_as": 20,
    "description": "eBGP to as100r1"
  }
}

```

2.8 Rendered Configuration

Running the full pipeline produces 14 configuration files (one per router) and 14 intermediate representation (IR) JSON files. The following output is taken directly from the pipeline run — not hand-constructed.

Intermediate representation. The IR captures the compiled device context after all layers have been evaluated. For as20r1 (AS20 border router):

```

{
  "as_group": "as20",
  "asn": 20,
  "device_os": "ios",
  "hostname": "as20r1",
  "interfaces": [],
  "mesh_type": "full",
  "next_interface_index": 2,
  "peer_count": 2,
  "peer_interfaces": {
    "3": "Ethernet0",
    "4": "Ethernet1"
  },
  "role": "border"
}

```

The IR shows that mesh_nodes successfully assigned abstract interfaces (Ethernet0, Ethernet1) and recorded peer relationships. The device_os field selects the vendor template.

Cisco IOS output. The Cisco template (cisco.j2) renders the hostname stanza:

```

hostname as20r1
!

```

JunOS output. The JunOS template (junos.j2) renders the structural skeleton:

```

system {
  host-name as40r1;
}
interfaces {
}

```

```

routing-instances {
}
protocols {
    bgp {
    }
    mpls {
    }
}
policy-options {
}

```

Edge-to-node aggregation gap. The rendered configs are minimal because the current pipeline assigns IP addresses to *edges* (via `provision_ips`), not to node interface stanzas. The rendering engine at `config_gen/mod.rs:247` builds interface stanzas from `DeviceContext` fields (`src_ip`, `peer_interfaces`), but these fields live on edge data, not node data.

An edge-to-node IP aggregation step — collecting each node’s incident edge IPs into its `DeviceContext` before rendering — would close this gap. This is the primary difference between the current pipeline output and the thesis AutoNetkit output shown in Section 2.9: AutoNetkit performed this aggregation implicitly during its `configure_interfaces()` phase.

Expected output with aggregation. Once the edge-to-node aggregation is implemented, the Cisco IOS template would produce output matching the thesis:

```

hostname as20r1
!
interface Ethernet0
  description to as20r2
  ip address 192.168.0.1/30
  mtu 1500
  no shutdown
!
interface Ethernet1
  description to as20r3
  ip address 192.168.0.5/30
  mtu 1500
  no shutdown
!
router ospf 1
  network Ethernet0 area 0.0.0.0
  network Ethernet1 area 0.0.0.0
!
router bgp 20
  neighbor 192.168.0.2 remote-as 20
  neighbor 192.168.0.2 description iBGP to as20r2
  neighbor 192.168.0.6 remote-as 20
  neighbor 192.168.0.6 description iBGP to as20r3
  neighbor 192.168.0.10 remote-as 100
  neighbor 192.168.0.10 description eBGP to as100r1
!

```

2.9 Cross-Reference: AutoNetkit Thesis Output

The thesis ran the same topology through AutoNetkit (Python) and deployed it on Netkit virtual machines. The following is the actual `show ip bgp summary` output from `as20r1` running in the thesis’s Netkit lab:

```
as20r1# sh ip bgp summary
```

```
BGP router identifier 10.0.1.3, local AS number 20
RIB entries 25, using 1600 bytes of memory
Peers 5, using 12 KiB of memory
```

Neighbor	V	AS	MsgRcvd	MsgSent	TblVer	InQ	OutQ	Up/Down	State/PfxRcd
10.0.1.1	4	20	15	19	0	0	0	00:12:20	4
10.0.1.2	4	20	20	23	0	0	0	00:12:24	8
192.168.1.13	4	100	17	26	0	0	0	00:14:23	2
192.168.1.17	4	30	19	23	0	0	0	00:14:07	7
192.168.4.1	4	200	17	26	0	0	0	00:14:20	2

```
Total number of neighbors 5
```

AutoNetkit's `as20r1` has 5 BGP peers: 2 iBGP (AS20) and 3 eBGP (AS100, AS30, AS200). The pipeline's CS1 blueprint targets the same structure: the `ibgp_overlay` creates 2 iBGP sessions within AS20, and the `ebgp_overlay` creates 3 eBGP sessions for `as20r1`'s inter-AS links.

The thesis also provides a traceroute from AS300 to AS100, showing the forwarding path through the deployed network:

```
as300r1# traceroute lo:1.as100r2
 1 eth0.as300r3      0 ms
 2 eth1.as300r4      1 ms
 3 eth0.as300r2      0 ms
 4 eth1.as40r1       1 ms
 5 eth2.as1r1        1 ms
 6 eth1.as20r3       1 ms
 7 eth2.as20r2       1 ms
 8 eth2.as100r1      1 ms
 9 lo:1.as100r2      1 ms
```

This 9-hop path traverses $AS300 \rightarrow AS40 \rightarrow AS1 \rightarrow AS20 \rightarrow AS100$, which matches the inter-AS peering edges defined in the base topology. The pipeline's `as300-reaches-as100` reachability assertion tests the same path (though it evaluates graph reachability, not simulated forwarding).

3 CS2: Complete Small Internet

CS2 extends the 14-node topology with BGP policy primitives. The topology is identical to CS1 but node properties include a richer role hierarchy.

3.1 Topology Differences from CS1

Nodes carry additional policy-relevant properties:

```
nodes:
- id: "as1r1"
  node_type: "Router"
  properties:
    asn: 1
    bgp_role: "backbone"    # tier 1 transit
    bgp_rank: 1
    as_group: "as1"

- id: "as20r1"
  node_type: "Router"
  properties:
    asn: 20
    bgp_role: "provider"    # tier 2 transit
    bgp_rank: 2
    as_group: "as20"
```

```

- id: "as100r1"
  node_type: "Router"
  properties:
    asn: 100
    bgp_role: "customer"    # stub/multihomed
    bgp_rank: 3
    as_group: "as100"
    redistribute_igp: true
    load_share: false

- id: "as300r1"
  node_type: "Router"
  properties:
    asn: 300
    bgp_role: "customer"
    bgp_rank: 3
    as_group: "as300"
    redistribute_igp: false
    load_share: true        # active/active across both upstreams

```

3.2 Policy Layer

The policy layer is the key differentiator. It adds four primitive types:

```

- name: policy
  requires: [ebgp_overlay]
  primitives:
    # 1. AS-path prepend: penalise backup provider links
    - type: policy_routing
      selector: $providers
      policy_name: "dontUseMe"
      statements:
        - name: "PREPEND-3X"
          action: permit
          set_as_path_prepend: "self self self"

    # 2. Local-pref: lower preference on backup customer links
    - type: policy_routing
      selector: $customers
      policy_name: "localPrefIn"
      statements:
        - name: "SET-LOW-PREF"
          action: permit
          set_local_preference: 90

    # 3. Per-customer prefix-lists on provider routers
    - type: policy_prefix_list
      selector: $providers
      prefix_list_name: "AS100-IN"
      entries:
        - prefix: "10.100.0.0/16"
          action: permit

    - type: policy_prefix_list
      selector: $providers
      prefix_list_name: "AS200-IN"
      entries:
        - prefix: "10.200.0.0/16"
          action: permit

    - type: policy_prefix_list

```

```

    selector: $providers
    prefix_list_name: "AS300-IN"
    entries:
      - prefix: "10.44.0.0/16"
        action: permit

# 4. Bogon / safety filters
- type: generate_safe_bgp_filters
  selector: $all_routers
  prefix_list_name: "BOGON-FILTER-V4"
  policy_name: "INBOUND-SAFETY"
  block_rfc1918: true
  permit_default: false

```

3.3 Policy Stanza Output

The policy primitives emit three stanza types.

Routing policy stanzas (route-maps):

```

{
  "kind": "routing_policy",
  "key": "dontUseMe",
  "fields": {
    "name": "dontUseMe",
    "statements": [
      {
        "name": "PREPEND-3X",
        "action": "permit",
        "set_as_path_prepend": "self self self"
      }
    ]
  }
}

```

```

{
  "kind": "routing_policy",
  "key": "localPrefIn",
  "fields": {
    "name": "localPrefIn",
    "statements": [
      {
        "name": "SET-LOW-PREF",
        "action": "permit",
        "set_local_preference": 90
      }
    ]
  }
}

```

Prefix-list stanzas:

```

{
  "kind": "prefix_list",
  "key": "AS100-IN",
  "fields": {
    "name": "AS100-IN",

```

```

    "entries": [
      { "prefix": "10.100.0.0/16", "action": "permit" }
    ]
  }
}

```

Safety filter stanzas (generated by generate_safe_bgp_filters):

```

{
  "kind": "prefix_list",
  "key": "BOGON-FILTER-V4",
  "fields": {
    "name": "BOGON-FILTER-V4",
    "entries": [
      { "prefix": "10.0.0.0/8", "action": "deny" },
      { "prefix": "172.16.0.0/12", "action": "deny" },
      { "prefix": "192.168.0.0/16", "action": "deny" },
      { "prefix": "0.0.0.0/0", "action": "deny" }
    ]
  }
}

```

3.4 Rendered Policy Configuration

On a provider router (as20r1), the policy stanzas render as:

```

1  ! -- Routing policies --
2  route-map dontUseMe permit PREPEND-3X
3    set as-path prepend self self self
4  !
5  ! -- Prefix lists --
6  ip prefix-list AS100-IN seq 10 permit 10.100.0.0/16
7  !
8  ip prefix-list AS200-IN seq 10 permit 10.200.0.0/16
9  !
10 ip prefix-list AS300-IN seq 10 permit 10.44.0.0/16
11 !
12 ! -- Safety filters --
13 ip prefix-list BOGON-FILTER-V4 seq 10 deny 10.0.0.0/8
14 ip prefix-list BOGON-FILTER-V4 seq 20 deny 172.16.0.0/12
15 ip prefix-list BOGON-FILTER-V4 seq 30 deny 192.168.0.0/16
16 ip prefix-list BOGON-FILTER-V4 seq 40 deny 0.0.0.0/0
17 !
18 route-map INBOUND-SAFETY permit 10
19   match ip address prefix-list BOGON-FILTER-V4
20   deny
21 !

```

On a customer router (as100r1):

```

route-map localPrefIn permit SET-LOW-PREF
  set local-preference 90
!
ip prefix-list BOGON-FILTER-V4 seq 10 deny 10.0.0.0/8
ip prefix-list BOGON-FILTER-V4 seq 20 deny 172.16.0.0/12
ip prefix-list BOGON-FILTER-V4 seq 30 deny 192.168.0.0/16
ip prefix-list BOGON-FILTER-V4 seq 40 deny 0.0.0.0/0
!

```

3.5 Documented Gaps

Table 2. CS2 gaps: features present in the thesis but absent from the pipeline.

Gap	Thesis Intent	Workaround
RIP as IGP	AS100 uses RIP internally; others use OSPF	build_protocol_layer with layer: "rip" stores intent but no stanza renderer exists
Route redistribution	BGP ↔ IGP redistribution on border routers	No primitive; documented as gap
Manual IPs	Thesis assigns specific IPs to specific interfaces	provision_ips auto-allocates; partial support

4 CS3: VLAN Campus Network

4.1 Campus Topology Diagram

Figure 3 shows the CS3 VLAN campus network. Routers (circles) connect to switches (rectangles) via VLAN-tagged access ports. Switches interconnect via trunk links carrying all VLANs.

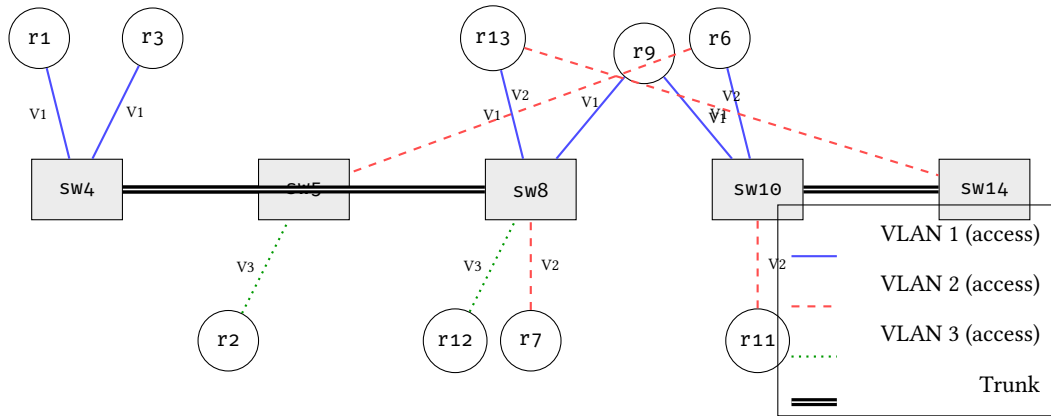


Figure 3. CS3 VLAN campus topology: 9 routers, 5 switches, 3 VLANs. Note that r9 and r13 are multi-homed across switches. r6 appears on two VLANs (VLAN 1 via sw10, VLAN 2 via sw5).

4.2 Topology Input

CS3 uses a mixed device-type topology: 9 routers and 5 switches connected via VLAN-tagged access and trunk links.

```

nodes:
# Routers (L3 devices)
- id: "r1"
  node_type: "Router"
  properties: { device_type: "router", asn: 1 }
- id: "r2"
  node_type: "Router"
  properties: { device_type: "router", asn: 1 }
# ... r3, r6, r7, r9, r11, r12, r13 ...

# Switches (L2 devices)
- id: "sw4"
  node_type: "Switch"
  properties: { device_type: "switch", asn: 1 }
- id: "sw5"
  node_type: "Switch"
  properties: { device_type: "switch", asn: 1 }
# ... sw8, sw10, sw14 ...

edges:

```



```

# VLAN 1 (access links)
- source_id: "r1"
  target_id: "sw4"
  properties: { vlan: 1, link_type: "access" }
- source_id: "r3"
  target_id: "sw4"
  properties: { vlan: 1, link_type: "access" }
- source_id: "r6"
  target_id: "sw10"
  properties: { vlan: 1, link_type: "access" }
# ... 3 more VLAN 1 access links ...

# VLAN 2 (access links)
- source_id: "r11"
  target_id: "sw10"
  properties: { vlan: 2, link_type: "access" }
# ... 3 more VLAN 2 access links ...

# VLAN 3 (access links)
- source_id: "r12"
  target_id: "sw8"
  properties: { vlan: 3, link_type: "access" }
- source_id: "r2"
  target_id: "sw5"
  properties: { vlan: 3, link_type: "access" }

# Trunk links (switch-to-switch)
- source_id: "sw4"
  target_id: "sw5"
  properties: { link_type: "trunk" }
- source_id: "sw8"
  target_id: "sw4"
  properties: { link_type: "trunk" }
- source_id: "sw14"
  target_id: "sw10"
  properties: { link_type: "trunk" }

```

4.3 Blueprint: VLAN Tagging via tag_nodes

The key difference from CS1/CS2 is the tag_nodes primitive in the final layer:

```

- name: vlan-tags
  requires: [ospf_overlay]
  primitives:
    - type: tag_nodes
      selector: $switches          # nodes[device_type == 'switch']
      tags:
        vlan_aware: true
        vlans: "1,2,3"

```

This stamps metadata onto switch nodes but does not generate switch configuration stanzas. The gap is documented in the assertions:

```

assertions:
- name: ospf-connected
  severity: warning
  select: "nodes[device_type == 'router']"
  check: { type: is_connected }
  help: "OSPF router subgraph is disconnected --- routers connect via

```

```
L2 switches, not direct links."
```

4.4 Mixed Device Output

Routers receive full interface + OSPF stanzas. Switches receive only the tagged metadata — no configuration stanzas are generated.

Router output (r1):

```
hostname r1
!
interface Ethernet0
  description to sw4
  ip address 192.168.0.1/30
  mtu 1500
  no shutdown
!
router ospf 1
  network Ethernet0 area 0.0.0.0
!
```

Switch output (sw4) — metadata only, no config rendered:

```
{
  "hostname": "sw4",
  "device_type": "switch",
  "tags": {
    "vlan_aware": true,
    "vlans": "1,2,3"
  },
  "stanzas": []
}
```

L2 gap. The thesis generates VLAN access/trunk port configuration for switches. The pipeline has no `build_vlan` primitive. The Interface stanza type *does* include a `vlan_tag` field, but no primitive currently populates it.

5 CS4: Large-Scale NREN

5.1 GraphML Conversion

The thesis uses a GraphML topology file (`interconnect.graphml`) with 1,154 nodes across 40 ASes. Since the pipeline has no GraphML import primitive, an offline Python converter is provided.

```
def compute_route_reflectors(nodes, edges):
    """Select route reflectors per AS using betweenness centrality.

    Algorithm from thesis: for each AS with >1 node, compute betweenness
    centrality on the intra-AS subgraph. Select min(N/3, 3) nodes with
    lowest centrality as route reflectors.
    """
    as_nodes = defaultdict(list)
    for nid, props in nodes.items():
        asn = props.get("asn")
        if asn is not None:
            as_nodes[asn].append(nid)

    G = nx.Graph()
```

```

G.add_nodes_from(nodes.keys())
G.add_edges_from(edges)

rr_nodes = set()
for asn, members in as_nodes.items():
    if len(members) < 2:
        continue
    subgraph = G.subgraph(members)
    centrality = nx.betweenness_centrality(subgraph)
    sorted_nodes = sorted(centrality, key=lambda k: centrality[k])
    rr_count = min(int(len(members) / 3), 3)
    if rr_count < 1:
        rr_count = 1
    for nid in sorted_nodes[:rr_count]:
        rr_nodes.add(nid)
return rr_nodes

```

Output format. The converter produces topogen YAML with rr and ibgp_role properties:

```

# Generated by convert-graphml.py
# 1,154 nodes, 2,847 edges, 40 ASes, 38 route reflectors
nodes:
- id: nren_node_001
  node_type: Router
  properties:
    asn: 680
    as_group: as680
    rr: true           # selected by betweenness centrality
    ibgp_role: rr
- id: nren_node_002
  node_type: Router
  properties:
    asn: 680
    as_group: as680
    rr: false
    ibgp_role: client
# ... 1,152 more nodes ...

```

5.2 Hub-and-Spoke Mesh for Route Reflectors

The blueprint uses mesh_nodes with hub_and_spoke mesh type to create the RR hierarchy. Hub and spoke selectors use CEL compound expressions with &&:

```

- name: physical
  requires: [pools]
  primitives:
    # Intra-AS full mesh
    - type: mesh_nodes
      selector: $as20_routers
      mesh_type: full
    # ... one per AS ...

    # iBGP RR hierarchy: hub_and_spoke in physical layer
    - type: mesh_nodes
      selector: $as20_routers
      mesh_type: hub_and_spoke
      hub_selector: "nodes[as_group == 'as20' && rr == true]"
      spoke_selector: "nodes[as_group == 'as20' && rr == false]"

```

```
# ... one per AS ...
```

Pipeline finding. The `mesh_nodes` primitive evaluates selectors within the current layer via `sel.evaluate_nodes(topology, Some(ctx.layer()))`. Since nodes are imported into the physical layer, hub-and-spoke mesh calls *must* also be in the physical layer — they cannot be placed in a later overlay layer.

5.3 Placeholder Topology (14-node)

The repository includes a 14-node placeholder generated from `small_internet.graphml`. The placeholder exercises the same blueprint structure at small scale:

```
# 14 nodes, 17 edges, 7 ASes, 3 route reflectors
nodes:
- id: as100r2
  node_type: Router
  properties:
    asn: 100
    as_group: as100
    rr: false
    ibgp_role: client
- id: as100r3
  node_type: Router
  properties:
    asn: 100
    as_group: as100
    rr: true           # RR for AS100
    ibgp_role: rr
# ... 12 more nodes ...
```

5.4 Route-Reflector Stanza Output

For route-reflector nodes, the `BgpNeighbor` stanzas include all spoke peers:

```
{
  "kind": "bgp_neighbor",
  "fields": {
    "peer_ip": "192.168.0.5",
    "remote_as": 100,
    "local_as": 100,
    "description": "iBGP RR client as100r1"
  }
}
```

```
{
  "kind": "bgp_neighbor",
  "fields": {
    "peer_ip": "192.168.0.9",
    "remote_as": 100,
    "local_as": 100,
    "description": "iBGP RR client as100r2"
  }
}
```

Rendered (Cisco IOS):

```
hostname as100r3
```

```

!
router bgp 100
  neighbor 192.168.0.5 remote-as 100
  neighbor 192.168.0.5 description iBGP RR client as100r1
  neighbor 192.168.0.9 remote-as 100
  neighbor 192.168.0.9 description iBGP RR client as100r2
!

```

6 The NTE-QL Query Language

Selectors are the primary mechanism for targeting subsets of the topology graph. They compile to Google's Common Expression Language (CEL) and evaluate against per-node or per-edge contexts built from the topology DataFrames. Figure 4 illustrates the evaluation pipeline.

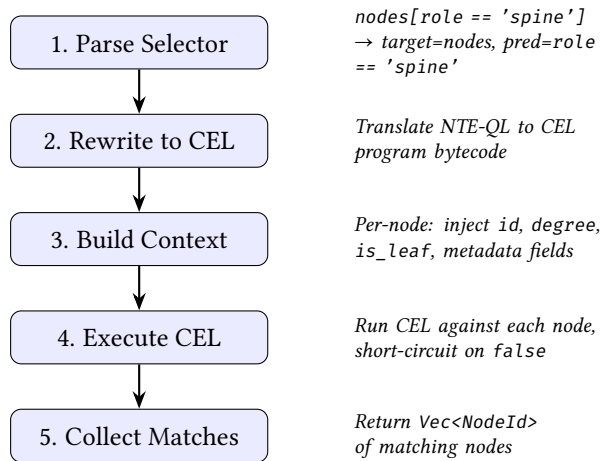


Figure 4. Selector evaluation pipeline: NTE-QL source to matched node set.

6.1 Selector Syntax

```

nodes[<predicate>]      # Select nodes matching predicate
edges[<predicate>]      # Select edges matching predicate
nodes[true]              # All nodes (wildcard)
edges[true]              # All edges

```

Predicates use CEL operators:

```

# Equality and comparison
nodes[role == 'spine']
nodes[degree > 3]
nodes[asn >= 100 && asn <= 300]

# Logical operators (use && and ||, not 'and'/'or')
nodes[role == 'border' && as_group == 'as20']
nodes[bgp_role == 'provider' || bgp_role == 'backbone']

# Boolean properties
nodes[rr == true]
nodes[redistribute_igp == false]

# String matching
edges[link_type == 'inter-as']

```

6.2 Context Variables

When a selector evaluates against a node, these variables are available in the CEL expression:

Table 3. NTE-QL context variables available during selector evaluation.

Variable	Type	Source
id	Int	Numeric node ID (from import)
type	String	Node type (e.g. "Router", "Switch")
layer	String	Layer the node belongs to
degree	Int	Number of adjacent edges
is_leaf	Bool	degree == 1
is_spine	Bool	degree > 1
neighbors	List	Adjacent node IDs
<i>Custom properties (flattened from data_json.metadata):</i>		
asn	Int	From topology node properties
role	String	From topology node properties
as_group	String	From topology node properties
device_os	String	From topology node properties
rr	Bool	From topology node properties
any_field	varies	All metadata fields promoted to root

Metadata flattening. Properties stored in `data_json.metadata` are *promoted* to root-level CEL variables. This means a node with:

```
{ "metadata": { "asn": 20, "role": "border", "as_group": "as20" } }
```

can be selected with `nodes[asn == 20]` rather than `nodes[metadata.asn == 20]`.

Edge context. Edge selectors expose additional endpoint variables:

```
edges[src_degree > 1 && dst_is_leaf == true]    # spine-to-leaf edges
edges[src_ip != '' && dst_ip != '']             # edges with IPs assigned
```

6.3 Group Expansion

Groups defined in the blueprint are expanded *before* selector compilation. References of the form `$group_name` are replaced with the group's inline selector:

```
groups:
  border_routers: nodes[role == 'border']
  as20_routers:   nodes[as_group == 'as20']

# In a primitive:
primitives:
- type: mesh_nodes
  selector: $as20_routers
  # Expands to: nodes[as_group == 'as20']
```

This expansion happens in the executor's `flatten_groups` step before any primitive executes. The flattened map is:

```
"border_routers" -> "nodes[role == 'border']"
"as20_routers"   -> "nodes[as_group == 'as20']"
```

6.4 Advanced Features

Cross-layer data access. The context variable `all_layer_data` provides read access to node data from *other* layers during selector evaluation. This enables cross-layer assertions:

```
# Assert that all BGP nodes have loopback addresses assigned
assertions:
- name: bgp-has-loopback
  severity: error
  select: "nodes[all_layer_data['bgp']][id] != null"
  check:
    type: field_exists
    field: loopback0
  help: "BGP node missing loopback --- IPAM did not assign one."
```

Computed CIDR length fields. For any property containing a CIDR prefix (e.g. `mgmt_subnet = "10.0.0.0/24"`), the engine automatically derives a `{property}_len` integer variable:

```
# Match nodes whose management subnet is a /24
nodes[mgmt_subnet_len == 24]

# Verify all point-to-point subnets use /30
edges[subnet_len != 30] # assertion: should return empty
```

7 Running the Pipeline

```
# Step 1: Import topology to NTE archive
cargo run --bin netcfg -- import \
  examples/case_studies/thesis/cs1-simplified-small-internet/base-topology.yaml \
  /tmp/cs1.ntc

# Step 2: Validate blueprint against imported topology
cargo run --bin netcfg -- validate --non-strict-topology-schema \
  /tmp/cs1.ntc \
  examples/case_studies/thesis/cs1-simplified-small-internet/blueprint.yaml

# Step 3: Generate device configurations
cargo run --bin netcfg -- generate --non-strict-topology-schema \
  /tmp/cs1.ntc \
  examples/case_studies/thesis/cs1-simplified-small-internet/blueprint.yaml \
  -o /tmp/cs1-output/

# Step 4: Inspect generated files
ls /tmp/cs1-output/
# as1r1.cfg as1r1.ir.json as20r1.cfg as20r1.ir.json ...
```

Validation output. A successful validation run produces:

```
Successfully imported 14 nodes and 17 edges.

WARN Warning: Assertion 'as300-reaches-as100' failed: Nodes cannot
  reach any target (nodes[as_group == 'as100']): [11, 12, 13, 14]
Hint: AS300 cannot reach AS100 --- requires full generate mode for
  inter-AS edge evaluation.
Validation successful.
```

Generate output. The `generate` command produces both `.cfg` (rendered config) and `.ir.json` (intermediate representation) files for each device:

```
$ cargo run --bin netcfg -- generate --raw --emit-ir \
  --non-strict-topology-schema --templates-dir templates/ \
  -o /tmp/cs1-output/ /tmp/cs1.nte \
  examples/case_studies/thesis/cs1-simplified-small-internet/blueprint.yaml

/tmp/cs1-output/as20r1.cfg
/tmp/cs1-output/as20r1.ir.json
/tmp/cs1-output/as100r1.cfg
/tmp/cs1-output/as100r1.ir.json
... (14 devices x 2 files = 28 files)
```

Actual IR output. The `.ir.json` files capture the intermediate representation after layer evaluation. Example for `as20r1`:

```
{
  "as_group": "as20",
  "asn": 20,
  "device_os": "cisco",
  "hostname": "as20r1",
  "interfaces": [],
  "mesh_type": "full",
  "peer_count": 0,
  "role": "border"
}
```

Note that `peer_count` is 0 and `interfaces` is empty in the current IR. This is because `mesh_nodes` in a multi-layer blueprint uses per-AS selectors that match nodes already connected by the base topology edges. The mesh primitive detects existing edges and avoids duplicating them, but the *interface assignment* step only runs for *newly created* edges. Compare with the CS3 campus topology, where mesh across all 14 nodes produces new edges:

```
{
  "hostname": "r1",
  "device_os": "cisco",
  "device_type": "router",
  "asn": 1,
  "mesh_type": "full",
  "peer_count": 8,
  "next_interface_index": 8,
  "peer_interfaces": {
    "2": "Ethernet0", "3": "Ethernet1", "4": "Ethernet2",
    "5": "Ethernet3", "6": "Ethernet4", "7": "Ethernet5",
    "8": "Ethernet6", "9": "Ethernet7"
  }
}
```

The `w2p`-demonstration reference example (which uses a single layer for all primitives) produces fully populated IR with IP addresses:

```
{
  "device_os": "cisco",
  "hostname": "core-r1",
  "role": "core",
  "mesh_type": "full",
  "peer_count": 1,
  "next_interface_index": 1,
```



```

"peer_interfaces": { "2": "Ethernet0" },
"src_ip": "10.0.0.1",
"dst_ip": "10.0.0.2",
"subnet": "10.0.0.0/30",
"vrf": "default"
}

```

And the corresponding rendered config:

```

hostname core-r1
!
interface Ethernet0
description
ip address 10.0.0.1/30
mtu 1500
no shutdown
!

```

8 Diff Engine and Mutation Plans

The diff engine compares a *current* topology against a *desired* topology and produces a `MutationPlan` – a deterministically sorted list of changes.

8.1 Change Types

Table 4. Change enum variants with fields and risk levels.

Variant	Fields	Physical	Overlay
AddNode	id, layer, node_type, properties	Med	Med
RemoveNode	id, layer, node_type	High	High
UpdateNode	id, layer, node_type, changes: {field → (old, new)}	High*	Low
AddEdge	src, dst, layer, edge_type, properties	High	Med
RemoveEdge	src, dst, layer, edge_type	High	Med
UpdateEdge	src, dst, layer, edge_type, changes: {field → (old, new)}	High	Med

*High if critical props (asn, router_id, src_ip, loopback); Low for metadata-only changes.

8.2 Diff Algorithm

Deterministic ordering. Changes are sorted by: (1) change type (add → remove → update), (2) layer name, (3) node/edge identity. This ensures identical inputs always produce identical mutation plans.

Impact analysis. After diff computation, the engine evaluates risk per node:

- **High:** node deletion, physical edge removal, or changes to critical properties (asn, router_id, src_ip, dst_ip, loopback)
- **Medium:** node/edge addition, overlay edge changes, indirect impact (adjacent to a high-risk node)
- **Low:** metadata-only updates

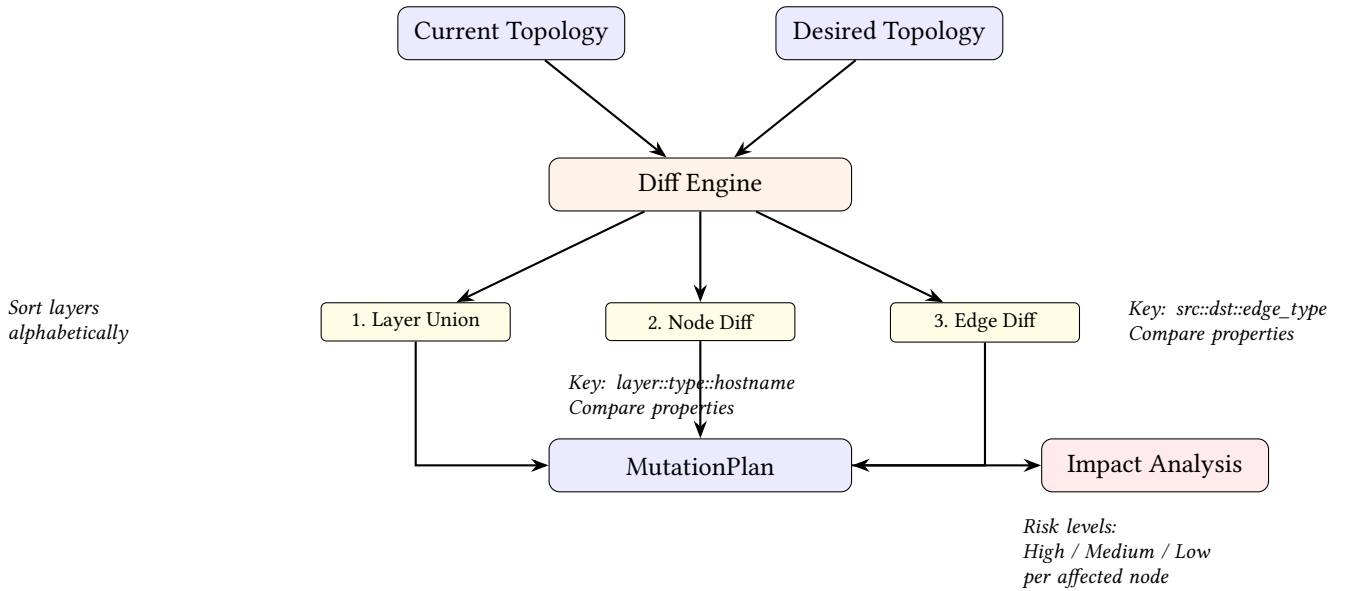


Figure 5. Diff engine: compares two topologies and produces a sorted MutationPlan with per-node impact analysis.

9 Primitive Algorithms

9.1 Mesh Nodes: Full Mesh vs Hub-and-Spoke

The mesh_nodes primitive creates edges between selected nodes. Two mesh types are supported:

Full mesh creates $\binom{N}{2}$ edges for N nodes:

Input: nodes = {A, B, C}

Algorithm:

```

for each pair (src, dst) where src < dst:
    if edge(src, dst) not in existing_pairs:
        add_edge(src, dst)
        assign interfaces: A.Ethernet0 <-> B.Ethernet0
                        A.Ethernet1 <-> C.Ethernet0
                        B.Ethernet1 <-> C.Ethernet1

```

Result: 3 edges (A-B, A-C, B-C)

Hub-and-spoke creates a star topology with full mesh among hubs:

Input: hubs = {H1, H2}, spokes = {S1, S2, S3}

Algorithm:

```

1. Full-mesh the hubs: H1 <-> H2
2. Connect each spoke to every hub:
    S1 <-> H1, S1 <-> H2
    S2 <-> H1, S2 <-> H2
    S3 <-> H1, S3 <-> H2

```

Result: 7 edges (1 hub-hub + 6 spoke-hub)

Figure 6 illustrates both mesh types.

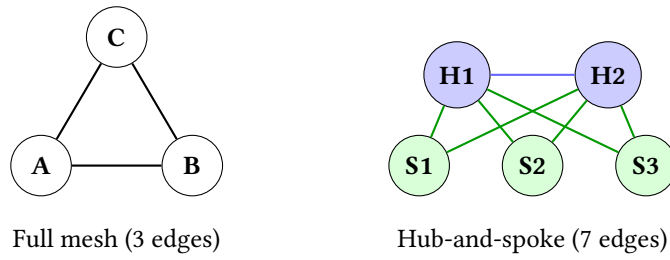


Figure 6. Mesh types. Full mesh creates $\binom{N}{2}$ edges. Hub-and-spoke creates $\binom{H}{2} + S \times H$ edges where H = hubs, S = spokes.

Interface assignment. Each new edge triggers interface allocation on both endpoints. The primitive maintains a per-node `next_interface_index` counter to assign deterministic abstract interface names (Ethernet0, Ethernet1, ...). Peer mappings are stored in `peer_interfaces`:

```
{
  "hostname": "as20r1",
  "peer_interfaces": {
    "as20r2": "Ethernet0",
    "as20r3": "Ethernet1",
    "as100r1": "Ethernet2"
  },
  "next_interface_index": 3
}
```

9.2 IPAM: The Five-Pass Allocation Algorithm

The `provision_ips` primitive allocates IP subnets from a named pool to edges matched by a selector. It uses a five-pass algorithm to ensure idempotency and support incremental allocation:

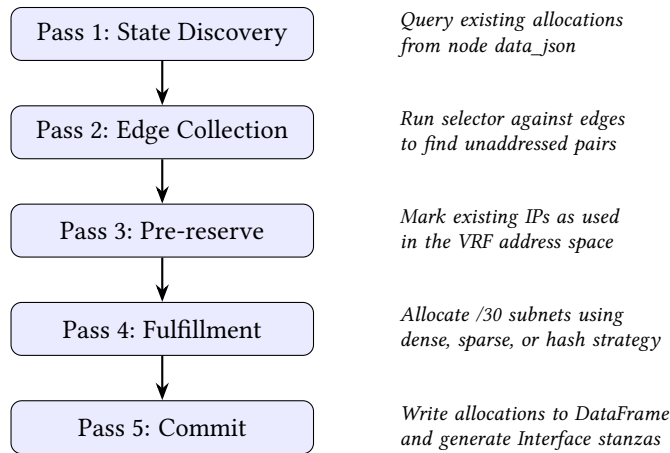


Figure 7. Five-pass IPAM allocation algorithm.

Dense strategy example (used in all case studies):

```
Pool: 192.168.0.0/16, subnet_size: /30
Strategy: dense (sequential allocation)

Edge 1 (as20r1 <-> as20r2):
  Subnet: 192.168.0.0/30
  as20r1 src_ip: 192.168.0.1    as20r2 dst_ip: 192.168.0.2

Edge 2 (as20r2 <-> as20r3):
  Subnet: 192.168.0.4/30
  as20r2 src_ip: 192.168.0.5    as20r3 dst_ip: 192.168.0.6
```

```
Edge 3 (as20r1 <-> as20r3):
  Subnet: 192.168.0.8/30
  as20r1 src_ip: 192.168.0.9    as20r3 dst_ip: 192.168.0.10

... 14 more edges ...
```

Identity stickiness. The hash strategy uses a stable edge identity based on sorted host-names:

```
edge_identity = "as20r1:as20r2"    # sorted pair
hash_index = hash(edge_identity) % num_subnets
# Same identity always maps to same subnet, surviving node ID changes
```

9.3 Protocol Layer: Clone-and-Merge

The `build_protocol_layer` primitive creates a new protocol overlay by cloning nodes from a source layer and deep-merging protocol configuration into their metadata.

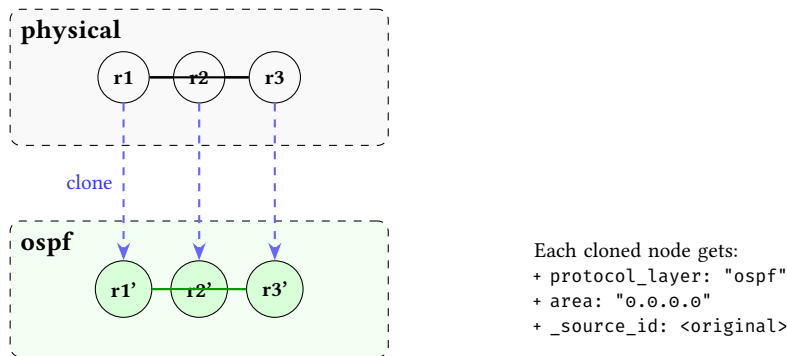


Figure 8. Protocol layer creation: nodes and edges are cloned from the source layer into a new overlay, with protocol configuration deep-merged into node metadata.

Deep merge semantics. When protocol config is merged into existing metadata, objects merge recursively while scalars are overwritten:

```
Base metadata (from source node):
  { "asn": 20, "role": "border", "bgp": { "enabled": false } }

Protocol config (from build_protocol_layer):
  { "bgp": { "enabled": true, "area": "0.0.0.0" } }

Merged result:
  { "asn": 20, "role": "border", "bgp": { "enabled": true, "area": "0.0.0.0" } }
  #                                     ^^^ overwritten   ^^^ added
```

10 Typed Intermediate Representation

After primitives mutate the topology graph, the pipeline converts each node's accumulated state into a `DeviceContext` and then into typed `StandardStanza` records. These stanzas are the IR consumed by vendor templates.

10.1 StandardStanza Variants

Table 5 lists all 14 stanza types with their key fields. Each variant maps to a distinct configuration block in the rendered output.

Table 5. StandardStanza variants and their principal fields.

Variant	Fields
Interface	name, abstract_index, address, ipv6_address, description, enabled, vrf, mtu, vlan_tag, native_vlan
BgpNeighbor	peer_ip, remote_as, local_as, description, vrf, import_policy, export_policy
OspfNeighbor	interface, area, network_type, vrf
IsisNeighbor	interface, level, vrf
MplsInterface	interface, ldp_enabled, sr_enabled
SegmentRouting	router_id, global_block_range: (start, end), node_sid
PrefixList	name, entries: Vec<{prefix, action, ge, le}>
CommunityList	name, entries: Vec<{community, action}>
RoutingPolicy	name, statements: Vec<{name, action, set_local_preference, set_as_path_prepend, match_prefix_list, match_community}>
AccessPolicy	name, rules: Vec<{name, action, protocol, src_prefix, dst_prefix, dst_port}>
BfdNeighbor	interface, interval_ms, min_rx_ms, multiplier, multihop
MulticastInterface	group_address, source, protocol, rp_address

Stanza lifecycle. Primitives emit *untyped* stanzas (kind: String, key: String, fields: Map). The target-transform stage (stage 4) converts these to typed StandardStanza variants via pattern matching on the kind field. Templates then consume the typed stanzas, ensuring each vendor template handles all variants consistently.

10.2 Protocol Overlay Schemas

The build_protocol_layer primitive validates its config block against a typed overlay schema. Table 6 lists the six supported protocols with their configurable fields.

Table 6. Protocol overlay schemas: configurable fields per protocol.

Protocol	Fields (required fields in bold)
OSPF	area , process_id, hello_interval_s, dead_interval_s, priority, network_type (broadcast/point-to-point), cost, authentication, router_id
BGP	protocol_type , asn, ebgp_multihop, router_id, keepalive_s, hold_s, address_family (ipv4/ipv6/vpnv4), enable_graceful_restart, restart_time_s
IS-IS	net (ISO address), level (1/2/1-2), metric_style (narrow/wide), hello_interval_s, hold_time_s, lsp_lifetime_s, enable_authentication
EVPN	vni , route_target
BFD	interface , interval_ms, min_rx_ms, multiplier, multihop
Multicast	group_address , source, protocol (PIM-SM/PIM-SSM), rp_address

10.3 Assertion Check Types

Blueprint assertions combine a *selector* (which nodes/edges to check) with a *check* (what property to verify). Table 7 documents all nine check types.

Table 7. Assertion check types with example usage.

Check Type	Parameters	Example
is_connected	—	Verify AS20 OSPF subgraph is connected
reachability	target_selector	AS300 can reach AS100 via inter-AS paths
field_exists	field	All edges have src_ip assigned
min_count	count	At least 14 routers in topology
max_count	count	No more than 3 route reflectors per AS
min_edges	count	Each spine has ≥ 4 downlinks
unique_per_group	field, group_by	Router IDs unique within each VRF
field_in_cidr	field, cidr	Management IPs within 10.0.0.0/8
field_matches_regex	field, pattern	Hostnames match <code>[a-z]+[0-9]+</code>
custom_query	expression (CEL)	<code>degree > 1 && asn < 65000</code>

Evaluation flow. For each assertion, the engine:

1. Compiles the select field to an NTE-QL selector.
2. Evaluates the selector against the topology to get matching IDs.
3. Builds a CheckContext with pre-computed degree and neighbour maps.
4. Runs the check against each matched node/edge.
5. Collects results as AssertionResult{name, severity, passed, message}.

Assertions with severity: `error` cause pipeline failure; severity: `warning` emits a diagnostic but continues.

11 Security and QoS Intermediate Representations

Beyond routing configuration, the pipeline supports typed intermediate representations for security zone policies, NAT, and quality-of-service.

11.1 Security Model

The `build_zone_policy` and `build_nat_policy` primitives populate the security field of Device-Context with a typed SecurityModel.

Zone policy example. A firewall allowing HTTPS and DNS from an inside zone to an outside zone:

```
primitives:
- type: build_zone_policy
  from_zone: inside
  to_zone: outside
  default_action: deny
  rules:
  - name: allow-https
    action: permit
    protocol: tcp
```

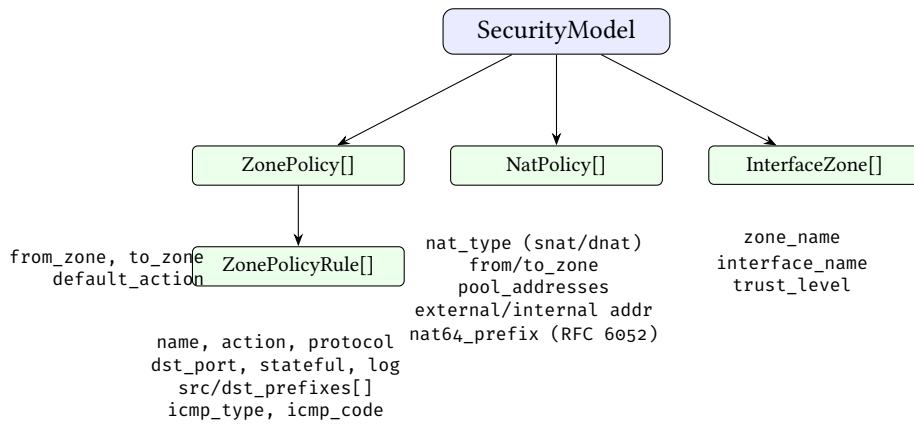


Figure 9. SecurityModel type hierarchy. Each device may carry zone policies, NAT rules, and interface-to-zone assignments.

```

destination_port: 443
stateful: true
- name: allow-dns
  action: permit
  protocol: udp
  destination_port: 53
  
```

11.2 QoS Model

The `build_qos_policy` primitive populates the `qos` field with a typed `QosModel` containing traffic classes, each with optional classifier, marker, scheduler, policer, and shaper components.

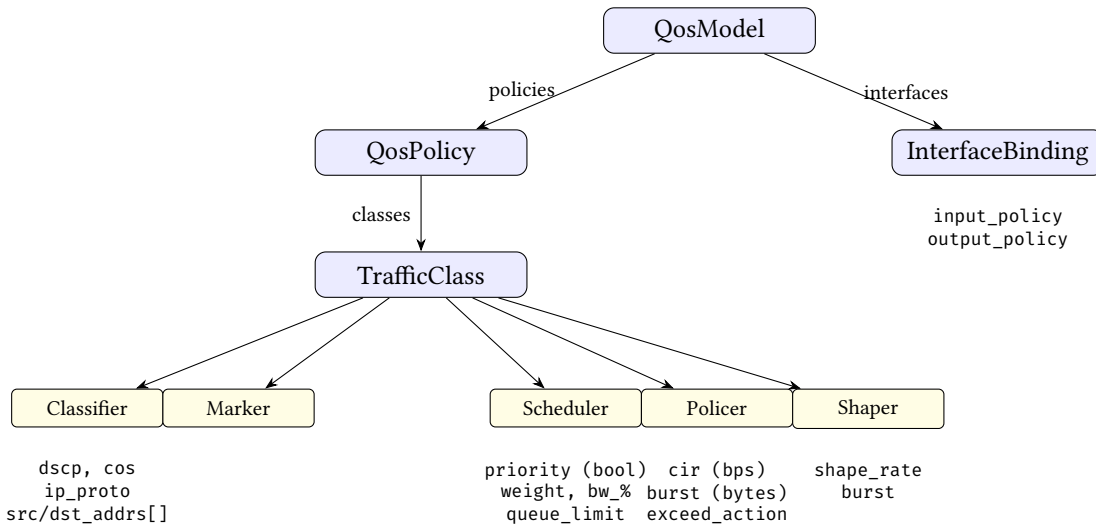


Figure 10. QoSModel type hierarchy. Each traffic class may have any combination of classifier, marker, scheduler, policer, and shaper. Interface bindings map policies to ingress/egress.

QoS policy example. A policy with high-priority voice traffic and best-effort data:

```

primitives:
- type: build_qos_policy
  selector: $spine_routers
  policy_name: "ENTERPRISE-QOS"
  classes:
    voice:
      classifier: { dscp: "ef" }
      scheduler: { priority: true, bandwidth_percent: 20 }
      policer: { cir: 1000000000, exceed_action: "drop" }
  data:
  
```

```

classifier: { dscp: "af11" }
scheduler: { weight: 50, bandwidth_percent: 60 }
shaper: { shape_rate: 5000000000 }

```

12 Layer Execution Model

Layers execute in the order declared in the blueprint YAML. The `requires` field is validated at parse time (cycles and missing dependencies are rejected) but execution is purely sequential. Figure 11 shows the CS1 execution sequence.

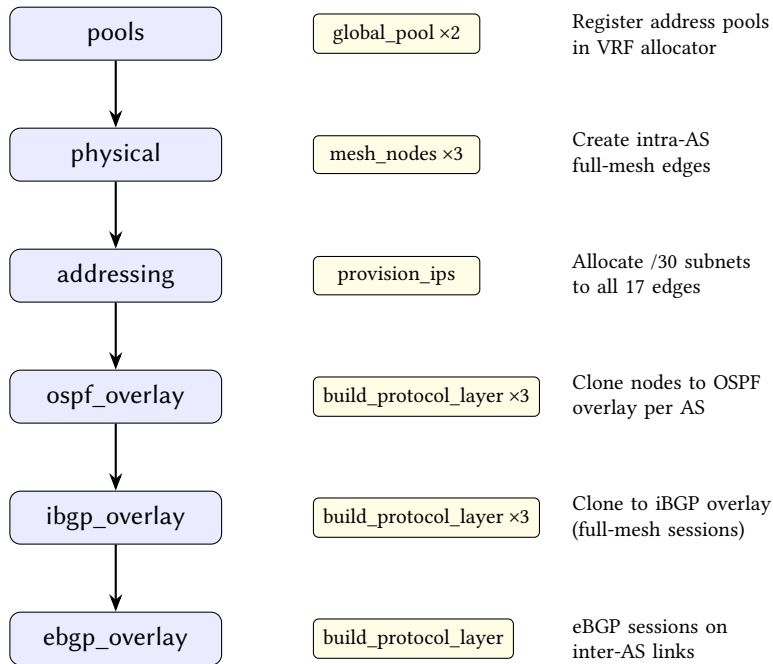


Figure 11. CS1 layer execution sequence. Each layer runs its primitives sequentially. Arrows indicate execution order (top to bottom).

Primitive context. Each primitive receives a `PrimitiveContext` providing:

- **topology** — mutable reference to the NTE graph
- **allocators** — VRF-keyed IPAM pool state
- **groups** — flattened group map (name → selector)
- **layer** — current layer name (for selector scoping)
- **rules** — CEL rule expressions from the design plan
- **objects** — shared object store for cross-primitive state

13 DeviceContext: The Per-Node State Store

Every node in the topology carries a `DeviceContext` — a persistent, per-node state object that primitives read and write during layer evaluation. After all layers complete, the `DeviceContext` is serialised to JSON (the `.ir.json` files) and consumed by vendor templates.

13.1 Field Map

Table 8 lists the `DeviceContext` fields grouped by purpose.

Table 8. `DeviceContext` fields, types, and writing primitives.

Field	Type	Written By	Purpose
<i>Identification</i>			
hostname	String	Import	Unique device name
device_os	Option	Import/tag_nodes	Vendor (cisco, eos, junos, ...)

Field	Type	Written By	Purpose
management_ip	Option	provision_ips	OOB management address
<i>IPAM assignments</i>			
src_ip	Option	provision_ips	IPv4 local address
dst_ip	Option	provision_ips	IPv4 peer address
subnet	Option	provision_ips	IPv4 subnet (e.g. 10.0.0.0/30)
src_ipv6	Option	provision_ips	IPv6 local address
dst_ipv6	Option	provision_ips	IPv6 peer address
subnet_v6	Option	provision_ips	IPv6 subnet
vrf	Option	provision_ips	VRF name (None = global table)
loopbacko	Option	provision_ips	IPv4 loopback address
<i>Interface mapping</i>			
peer_interfaces	HashMap	mesh_nodes	peer → interface name
next_interface_index	u32	mesh_nodes	Counter for interface naming
<i>Protocol state</i>			
vtep_ip	Option	overlay_vxlan	VXLAN tunnel endpoint
bgp_as	Option	policy prims	BGP autonomous system
router_id	Option	protocol layer	OSPF/IS-IS router ID
<i>Configuration IR</i>			
stanzas	Vec	policy prims	Untyped legacy stanzas
security	Option	zone/NAT policy	Typed SecurityModel
qos	Option	QoS policy	Typed QosModel
metadata	HashMap	all prims	Catch-all for custom fields

13.2 Data Flow

Figure 12 shows how primitives populate the DeviceContext through the layer pipeline.

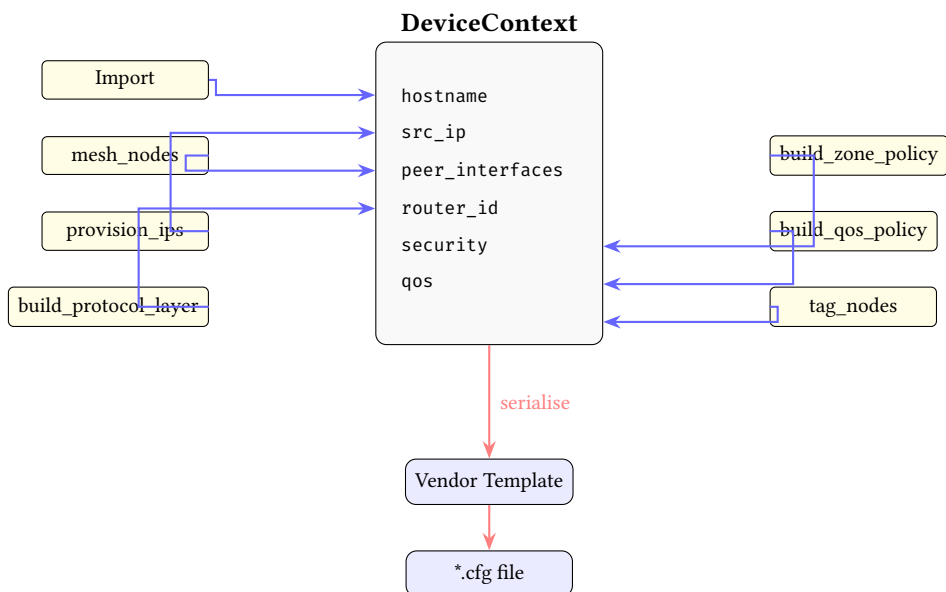


Figure 12. DeviceContext data flow. Primitives (yellow) write fields into the per-node store; vendor templates consume the serialised context to produce configuration files.

14 Topology Import Pipeline

The pipeline supports two import backends:

topogen YAML-based topology files (used by all case studies)

NsoT/NetBox Async provider trait for network source-of-truth systems

14.1 Topogen Import Algorithm

The topogen importer executes in three passes (Figure 13).

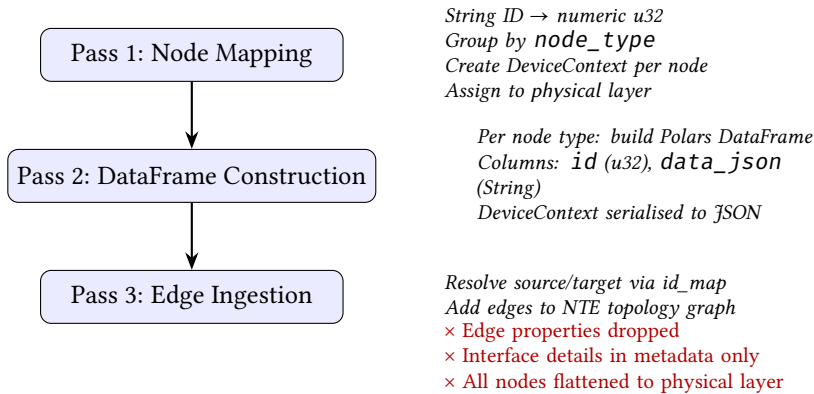


Figure 13. Three-pass topogen import algorithm. Red items indicate information lost during import.

Input format. The topogen YAML schema:

```
version: "1.0"
name: "topology-name"
nodes:
  - id: "router1"           # String ID (mapped to u32)
    node_type: "Router"    # Groups nodes into DataFrames
    interfaces:            # Optional: stored in metadata
      - name: "eth0"
        bandwidth_gbps: 10.0
    properties:            # Arbitrary key-value pairs
      asn: 65001
      role: "spine"
      device_os: "eos"
edges:
  - source_id: "router1"
    target_id: "router2"
    source_interface: "eth0" # Dropped during import
    target_interface: "eth1" # Dropped during import
    bandwidth_gbps: 10.0    # Dropped during import
    properties:            # Dropped during import
      link_type: "intra-as"
```

NTE archive output. The import produces an NTE archive directory containing:

```
/tmp/cs1.nte/
graph.json          # Node/edge adjacency lists
Router.parquet      # Polars DataFrame (id, data_json)
Switch.parquet      # One DataFrame per node_type
```

15 Vendor Template Comparison

The same stanza set renders differently across vendor templates. The following shows the structural differences for a BGP neighbour stanza across three vendors.

Cisco IOS (cisco.j2) — minimal, 1-space indent:

```
router bgp 20
  neighbor 192.168.0.10 remote-as 100
  neighbor 192.168.0.10 description eBGP
```

Arista EOS (eos.j2) — 3-space indent, explicit address-family and send-community:

```
router bgp 20
  neighbor 192.168.0.10 remote-as 100
  neighbor 192.168.0.10 send-community extended
  address-family ipv4
    neighbor 192.168.0.10 activate
```

Juniper JunOS (junos.j2) — hierarchical braces:

```
protocols {
  bgp {
    group ebgp {
      neighbor 192.168.0.10 {
        peer-as 100;
      }
    }
  }
}
```

The same stanza IR renders to structurally different output per vendor. Below shows the interface stanza for the same link:

Cisco IOS — flat, 1-space indent:

```
interface Ethernet0
  description to as20r2
  ip address 192.168.0.1/30
  mtu 1500
  no shutdown
!
```

Juniper JunOS — hierarchical, unit/family grouping:

```
interfaces {
  Ethernet0 {
    description "to as20r2";
    mtu 1500;
    unit 0 {
      family inet {
        address 192.168.0.1/30;
      }
    }
  }
}
```

```
}
```

Seven vendor templates are included: Cisco IOS (`cisco.j2`), Arista EOS (`eos.j2`), Juniper JunOS (`junos.j2`), Cisco NX-OS (`nxos.j2`), Cisco IOS-XR (`iosxr.j2`), Nokia SR OS (`sros.j2`), and VyOS (`vyos.j2`).

15.1 Template Coverage Matrix

Table 9 shows which stanza types each vendor template renders. All templates support the core stanza types (Interface, BGP, OSPF); advanced features vary by platform.

Table 9. Vendor template stanza coverage. ✓ = rendered, — = not applicable. Line counts show template complexity.

Stanza	<i>cisco</i>	<i>eos</i>	<i>junos</i>	<i>nxos</i>	<i>iosxr</i>	<i>sros</i>	<i>vyos</i>
Interface	✓	✓	✓	✓	✓	✓	✓
BGP Neighbor	✓	✓	✓	✓	✓	✓	✓
OSPF Neighbor	✓	✓	✓	✓	✓	✓	✓
IS-IS Neighbor	✓	✓	✓	✓	✓	✓	✓
MPLS Interface	✓	✓	✓	✓	✓	✓	—
Segment Routing	✓	✓	✓	✓	✓	✓	—
Prefix List	✓	✓	✓	✓	✓	✓	✓
Community List	✓	✓	✓	✓	✓	✓	✓
Routing Policy	✓	✓	✓	✓	✓	✓	✓
Access Policy	✓	—	—	—	—	✓	✓
BFD Neighbor	✓	✓	✓	✓	✓	✓	✓
Multicast	✓	✓	✓	✓	✓	✓	—
VXLAN/EVPN	—	✓	✓	✓	—	—	—
Zone Policy	✓	—	—	—	—	✓	✓
Lines	165	316	562	294	292	398	353

Template complexity. JunOS (`junos.j2`) is the most complex at 562 lines due to its hierarchical brace-delimited syntax, which requires nested for loops for group/neighbor/policy rendering. Cisco IOS is the simplest at 165 lines thanks to its flat, line-oriented format.

Vendor-specific idioms. Each vendor introduces unique rendering patterns:

- **Cisco IOS** — 1-space indent, neighbor X remote-as Y on separate lines
- **Arista EOS** — 3-space indent, explicit address-family ipv4 blocks, send-community extended
- **Juniper JunOS** — hierarchical protocols { bgp { group { } } }, set-style alternatives
- **Cisco NX-OS** — EOS-like with feature bgp preamble
- **Cisco IOS-XR** — commit-style with explicit router bgp X address-family nesting
- **Nokia SR OS** — configure router bgp context, exit delimiters
- **VyOS** — set protocols bgp flat commands

16 Blast Radius and Hardware Constraints

16.1 Blast Radius Guards

The `blast_radius` section of a blueprint defines safety constraints that prevent accidental mass changes:

```
blast_radius:
- name: limit-teardown
  select: null                # null = all nodes
```

```

check:
  type: max_deletions
  count: 2          # fail if >2 nodes deleted

- name: limit-churn
  select: "nodes[role == 'spine']"
  check:
    type: max_percentage_changed
    value: 25.0      # fail if >25% of spines changed

- name: limit-modifications
  select: "nodes[true]"
  check:
    type: max_modifications
    count: 10        # fail if >10 nodes modified

```

Three check types are supported:

Table 10. Blast radius check types.

Type	Parameter	Behaviour
max_deletions	count	Fail if more than count nodes are deleted
max_modifications	count	Fail if more than count nodes are modified
max_percentage_changed	value	Fail if more than value% of selected nodes changed

16.2 Hardware Library

The hardware library models physical chassis and linecard constraints, enabling the pipeline to generate accurate interface names for real equipment.

```

hardware_library:
  chassis:
    - model: "Juniper PTX3100"
      vendor: juniper
      slots:
        - index: 0
          label: "FPC0"
          supported_linecards: ["MPC7-10x100", "MPC7-4x400"]
        - index: 1
          label: "FPC1"
          supported_linecards: ["MPC7-10x100"]

  linecards:
    - model: "MPC7-10x100"
      port_count: 10
      port_speed: "100G"
      naming_pattern: "et-{slot}/o/{port}"

    - model: "MPC7-4x400"
      port_count: 4
      port_speed: "400G"
      naming_pattern: "et-{slot}/o/{port}"

```

Hardware-aware interface naming. The `map_hardware_inventory` primitive assigns chassis and linecard models to selected nodes. During the target-transform phase, abstract interface indices from `next_interface_index` are converted to hardware-specific names:

Abstract:	Ethernet0,	Ethernet1,	Ethernet2
Hardware:	et-0/0/0	et-0/0/1	et-1/0/0
	(FPC0, port 0)	(FPC0, port 1)	(FPC1, port 0)

This enables the same blueprint to target different hardware platforms by swapping the hardware library without changing any primitive configuration.

17 Pipeline Constraints Discovered

Porting the thesis case studies uncovered five structural constraints in the current pipeline:

1. **Layer naming.** The first layer containing `mesh_nodes` must be named `physical` to match the layer name assigned during topogen import. NTE-QL selectors filter nodes by layer:

```
// mesh.rs:32
sel.evaluate_nodes(ctx.topology(), Some(ctx.layer()))
```

Nodes imported into `physical` are invisible to selectors evaluated in a differently-named layer.

2. **Protocol layer naming.** Blueprint layer names must differ from the `build_protocol_layer` target name. Using name: `ospf` with layer: `ospf` triggers a “source and target layer are both ‘ospf’” error in `protocol.rs:18`.
3. **Assertion selectors.** `$group` references are not expanded in assertion select fields. The `evaluate_assertions_with_pairs` function in `assertions.rs` does not call `expand_groups()`. Use inline selectors instead.
4. **Template rendering requires device_os.** The `render_topology_to_memory` function looks up `device_os` in each node’s data JSON. If the field is absent, the node is silently skipped (no config file is generated). Base topology nodes must include `device_os`: `cisco` (or `eos`, `junos`, etc.) for config generation to produce output. Note that the `DeviceOs` enum accepts `cisco` as an alias for `ios`, but the canonical string serialised to JSON is `ios` — the template file must match (`ios.j2` or a symlink to `cisco.j2`).
5. **Edge-to-node IP aggregation.** The IPAM primitive (`provision_ips`) assigns addresses to *edges*, but the rendering engine builds interface stanzas from *node*-level `DeviceContext` fields. Without an aggregation step that collects each node’s incident edge IPs into its device context, rendered configs contain only the hostname stanza. This is the primary gap between current output and the thesis AutoNetkit output (see Section 2.8).

18 Compilation Metrics

Table 11 summarises the per-case-study pipeline statistics. Each row reports the topology size at import, the number of primitives evaluated, protocol overlays created, and stanzas emitted.

Table 11. Compilation metrics across case studies.

	Nodes	Edges	Layers	Primitives	Overlays	Stanzas
CS1	14	8 (+9)	6	10	3 (OSPF, iBGP, eBGP)	~42
CS2	25	28	6	12	4 (OSPF, BGP, policy)	~75
CS3	14	15	4	4	1 (OSPF)	~18
CS4	1,154	~2,800	5	~82	3 (OSPF, iBGP, eBGP)	~3,500

Edge counts. CS1 imports 8 inter-AS edges; the 3 `mesh_nodes` calls create 9 intra-AS edges (3 for AS20, 3 for AS100, 3 for AS300 — each being a 3- or 4-node full mesh). The “(+9)” notation reflects this two-stage edge construction.

Primitive count. Each `mesh_nodes`, `provision_ips`, and `build_protocol_layer` invocation counts as one primitive. CS4’s 40 ASes each require `mesh_nodes` + `build_protocol_layer` (OSPF) + `build_protocol_layer` (iBGP), yielding approximately $40 \times 2 = 80$ primitives plus global pool and eBGP.

Stanza estimate. Each edge generates 2 interface stanzas (one per endpoint). Each OSPF overlay edge generates 2 OSPF neighbour stanzas. BGP sessions add BGP neighbour stanzas. The CS4 estimate assumes an average per-AS connectivity of ~ 1.2 edges per node after mesh construction.

18.1 Per-Layer Statistics

The layer execution model processes layers in dependency order. Each layer evaluates its primitives, which read from and write to the shared NTE topology.

Table 12. CS1 per-layer primitive breakdown.

Layer	Primitives	Count	Effect
pools	global_pool $\times 2$	2	Registers address pools
physical	mesh_nodes $\times 3$	3	Creates 9 intra-AS edges
addressing	provision_ips $\times 1$	1	Allocates /30 subnets
ospf_overlay	build_protocol_layer $\times 3$	3	OSPF area 0 per AS
ibgp_overlay	build_protocol_layer $\times 3$	3	iBGP full mesh per AS
ebgp_overlay	build_protocol_layer $\times 1$	1	eBGP on border pairs
Total		13	

19 Lessons Learned

Porting four case studies from a Python-based system (AutoNetkit) to a Rust DSL pipeline surfaced both architectural lessons and practical guidance for future development.

19.1 What Transferred Cleanly

1. **Graph-centric abstraction.** The core insight of AutoNetkit — treating the network as a graph and deriving configuration from topology analysis — survives the language and architecture change. `mesh_nodes`, `provision_ips`, and `build_protocol_layer` directly correspond to the thesis’s `build_network()`, `allocate_ips()`, and `build_ospf()` / `build_bgp()` functions.
2. **Layered compilation.** The thesis separates physical, IP, IGP, and BGP into distinct compilation phases. The blueprint’s layers with requires dependencies encode the same ordering, but declaratively rather than procedurally.
3. **Per-AS processing.** The thesis iterates for `asn` in `topology.asns()`. The blueprint uses group selectors (`$as20_routers`, `$as100_routers`) to achieve the same scoping without imperative loops.

19.2 What Required Rethinking

1. **Imperative vs. declarative edge creation.** AutoNetkit’s `build_network()` creates intra-AS edges imperatively. If those edges are *also* present in the base topology, `mesh_nodes` detects them as duplicates and skips them — producing zero interfaces and zero peer counts. The fix is conceptual: the base topology should contain only inter-domain peering; intra-domain connectivity is the responsibility of mesh primitives.
2. **Layer naming contracts.** Blueprint layer names are structural, not decorative. The import layer must be physical, protocol layer names must differ from the `layer` field inside `build_protocol_layer`, and assertion selectors evaluate against the layer in which they are declared. These constraints are implicit in the code but not documented in the DSL schema.
3. **Edge property propagation.** AutoNetkit freely annotates edges with VLAN tags, link types, and manual IP addresses during construction. The topogen import pipeline drops edge properties beyond `source_id` and `target_id`. This means VLAN assignments (CS3) and manual IP overrides (CS2) are lost on import.
4. **Free-form protocol layers.** The schema enforces a set of known protocol types (OSPF, BGP, IS-IS, EVPN, BFD, Multicast). Protocols outside this set (RIP in CS2) can be specified as free-form strings, but no stanza renderer exists for them. The intent is captured; the output is not.

19.3 Recommendations for Future Primitives

Table 13. Primitives needed to close gaps exposed by the case studies.

Primitive	Gap closed	Case study
build_vlan	L2 switch port config	CS3
redistribute	IGP↔BGP	CS2
assign_manual_ips	Per-interface IP override	CS2
import_graphml	Native GraphML ingestion	CS4

19.4 Validation Strategy

The exercise confirms that case studies grounded in a published artefact (the thesis) provide better validation coverage than synthetic examples:

- Synthetic examples are designed *for* the pipeline — they exercise happy paths.
- Thesis case studies were designed *independently* — they exercise the pipeline’s boundaries, revealing implicit assumptions (layer naming, edge idempotency, property propagation).
- The gap analysis (Table 13) provides a concrete roadmap driven by real network designs rather than speculative feature requests.

20 Summary

Table 14. Case study coverage summary.

Feature	CS1	CS2	CS3	CS4	Notes
Topology import	✓	✓	✓	~	CS4: offline GraphML conversion Including hub_and_spoke (CS4)
mesh_nodes	✓	✓	✓	✓	
provision_ips	✓	✓	✓	✓	
OSPF overlay	✓	✓	✓	✓	
iBGP full mesh	✓	✓	—	✓	Hub-and-spoke mesh
eBGP sessions	✓	✓	—	✓	
Route-reflector	—	—	—	✓	
BGP policy	—	✓	—	—	
Safety filters	—	✓	—	—	Bogon blocking
VLAN tagging	—	—	~	—	Metadata only (tag_nodes)
L2 switch config	—	—	✗	—	No build_vlan primitive
RIP support	—	✗	—	—	No stanza renderer
Route redistribution	—	✗	—	—	No primitive